UNIVERSIDAD MANUELA BELTRÁN

DESCRIPCIÓN Y FORMULACIÓN DEL PROYECTO

PlanifiCPU · Sistemas Operativos · Grupo G11 · 2026

Equipo: David Zambrano, Daniel Pérez, Axel Salgado

1. Nombre del proyecto

PlanifiCPU: simulador de algoritmos de planificación de procesos en la CPU.

2. Descripción general del producto

PlanifiCPU es una aplicación de escritorio desarrollada en Python que simula los algoritmos de planificación de CPU FCFS, SJF (con y sin expropiación), Round Robin y por Prioridad (con y sin expropiación). A partir de una lista de procesos con su tiempo de llegada, ráfaga de CPU y prioridad, genera el diagrama de Gantt de la ejecución, registra cada cambio de contexto, calcula los tiempos de espera, retorno y respuesta, y compara el desempeño de los cuatro algoritmos con el mismo conjunto de procesos. Los escenarios y el historial de simulaciones se guardan en una base de datos SQLite local.

3. Levantamiento de información

Contexto

En la asignatura de Sistemas Operativos, la planificación de CPU se estudia resolviendo ejercicios en papel: se entrega una tabla de procesos con su tiempo de llegada, ráfaga de CPU y prioridad, y el estudiante debe dibujar el diagrama de Gantt y calcular los tiempos de espera, retorno y respuesta para cada algoritmo.

Técnicas utilizadas

Técnica	Qué se hizo	Qué se obtuvo
Revisión documental	Lectura de la bibliografía del curso sobre planificación de CPU.	Definición de los algoritmos, fórmulas de las métricas y convenciones de desempate.
Análisis de ejercicios del curso	Revisión del formato de los ejercicios que se resuelven a mano (PID, llegada, ráfaga, prioridad).	Formato de entrada del simulador y del archivo CSV.
Identificación de usuarios	Se identificó quién usa la herramienta y para qué.	Dos actores: estudiante y docente.
Ficha técnica del proyecto	Definición de objetivo, funcionalidad, entradas, procesamiento, salidas y alcance.	Requerimientos funcionales y no funcionales.

4. Planteamiento de la necesidad o problemática

Problemática

La planificación de CPU es un tema abstracto y en el curso se resuelve a mano en papel. Esto genera tres dificultades: (1) no hay una forma rápida de verificar si el diagrama de Gantt y los tiempos calculados son correctos, y un error en un tramo se arrastra a todas las métricas; (2) comparar algoritmos es lento, porque

hay que repetir el ejercicio completo cuatro veces con los mismos datos; y (3) en papel no se ve la ejecución en el tiempo, por lo que no es evidente cuándo ocurre una expropiación o un cambio de contexto, que es justamente lo que diferencia a los algoritmos.

Necesidad

Se necesita una herramienta que permita ver y comparar de forma visual la planificación de CPU: ingresar los procesos una sola vez, obtener el diagrama de Gantt con sus métricas, comparar los cuatro algoritmos y guardar los escenarios para volver a ellos.

5. Justificación de la solución

- Convierte un tema abstracto en algo visible: el diagrama de Gantt y el registro de ejecución muestran en qué momento ocurre cada despacho, expropiación y cambio de contexto.
- Permite verificar los ejercicios resueltos a mano: el motor sigue las mismas convenciones del curso y sus resultados se validan con pruebas automáticas sobre ejercicios de resultado conocido.
- Ahorra tiempo al comparar: con un clic se ejecutan los cuatro algoritmos sobre los mismos procesos y se ve cuál ofrece menor tiempo de espera, retorno o respuesta.
- Es viable para el equipo: usa Python y herramientas incluidas en él (Tkinter, SQLite), no requiere servidor ni internet y funciona en los equipos con Windows de estudiantes y docente.

6. Objetivo general

Desarrollar una aplicación de escritorio en Python que simule los algoritmos de planificación de CPU FCFS, SJF, Round Robin y por Prioridad, para visualizar mediante diagramas de Gantt cómo el sistema operativo asigna el procesador a los procesos y comparar su desempeño con métricas de tiempo.

7. Objetivos específicos

1. Implementar los algoritmos FCFS, SJF (con y sin expropiación), Round Robin y Prioridad (con y sin expropiación).
2. Generar el diagrama de Gantt y el registro de cambios de contexto de cada simulación.
3. Calcular los tiempos de espera, retorno y respuesta por proceso y en promedio.
4. Comparar los cuatro algoritmos con el mismo conjunto de procesos mediante una tabla y una gráfica.
5. Guardar escenarios y resultados en archivos CSV y en una base de datos SQLite local.

8. Alcance inicial del producto

Incluye

- 4 algoritmos: FCFS, SJF, Round Robin y Prioridad (SJF y Prioridad con y sin expropiación).
- Diagrama de Gantt, registro de ejecución y métricas por proceso.
- Comparación de los 4 algoritmos con el mismo conjunto de procesos.
- Carga y exportación de escenarios y resultados en CSV; Gantt en PNG.

No incluye

- Planificar procesos reales del sistema operativo.
- Sistemas multiprocesador o multinúcleo.
- Operaciones de E/S ni gestión de memoria.
- Versión web o móvil.
- Usuarios, login o servidor de base de datos.

- Base de datos SQLite local para escenarios e historial de simulaciones.
- Código en GitHub, pruebas automáticas y manual de uso (README).

9. Usuarios o beneficiarios

Usuario	Tipo	Necesidad que atiende
Estudiante de Sistemas Operativos	Beneficiario principal	Comprobar sus ejercicios resueltos a mano y entender por qué un algoritmo da mejores tiempos que otro.
Docente de Sistemas Operativos	Usuario	Mostrar en clase la ejecución paso a paso y preparar ejemplos con resultado verificable.
Equipo de desarrollo (G11)	Beneficiario indirecto	Aplicar en un producto real los conceptos de planificación vistos en el curso.

10. Funcionalidades principales

Funcionalidad	Descripción
Registrar procesos	PID, tiempo de llegada, ráfaga de CPU y prioridad; editar y eliminar.
Elegir algoritmo	FCFS, SJF, Round Robin (con quantum) o Prioridad; modo expropiativo en SJF y Prioridad.
Generar el diagrama de Gantt	Tramos de ejecución, CPU ociosa y registro de cada cambio de contexto.
Calcular métricas	Tiempo de espera, de retorno y de respuesta por proceso y sus promedios.
Comparar algoritmos	Los 4 algoritmos con el mismo conjunto de procesos, en tabla y gráfica.
Cargar y exportar	Escenarios en CSV, resultados en CSV y el Gantt en PNG.
Historial	Escenarios y simulaciones guardados en SQLite, con opción de reabrirlos.

11. Relación con los conceptos de Sistemas Operativos

Concepto	Cómo se aplica en PlanifiCPU
Planificador de corto plazo (despachador)	El motor decide en cada instante qué proceso de la cola de listos pasa a ejecución.
Estados de un proceso	Nuevo (registro), listo (cola de listos), ejecución (tramo del Gantt) y terminado.
Algoritmos de planificación	FCFS, SJF/SRTF, Round Robin y Prioridad, con sus reglas de selección.
Expropiación	Modo no expropiativo vs. expropiativo; quantum en Round Robin.
Cambio de contexto	Se cuenta y se registra cada vez que la CPU pasa a otro proceso.
Métricas de planificación	Tiempo de espera, retorno y respuesta; tiempos en que la CPU queda ociosa.

Inanición	Se puede observar en Prioridad y SJF cuando llegan de forma continua procesos más prioritarios o más cortos.



Figura 1. Ciclo de vida de un proceso en el simulador.

12. Estado actual y próximos avances

Prototipo funcional versión 1.0. Ya están implementados los cuatro algoritmos (con y sin expropiación), el diagrama de Gantt con registro de ejecución, las métricas, la comparación de algoritmos, la carga y exportación en CSV/PNG, la base de datos SQLite con historial y 13 pruebas automáticas del motor que pasan. Aunque la guía no exige el producto implementado en esta primera entrega, el equipo adelantó la implementación para validar la arquitectura propuesta.

- Modo paso a paso: avanzar la simulación unidad por unidad mostrando la cola de listos en cada instante.
- Ampliar las pruebas automáticas con casos límite (procesos con la misma llegada, CPU ociosa larga, quantum grande).
- Validar el simulador con los ejercicios propuestos por el docente en clase.
- Manual de usuario con capturas en la carpeta documentacion.
- Empaquetar la aplicación como ejecutable .exe para Windows (PyInstaller).

13. Equipo

Integrante	Usuario de GitHub
David Zambrano	daviid0611
Daniel Pérez	monarca1290
Axel Salgado	AxelN28

Repositorio: `https://github.com/daviid0611/PlanifiCPU_SistemasOperativos_G11`